WARWICK

SCHOOL OF ENGINEERING

Lab 4 – PCam 5C Demo

Module Code:

ES4F3/ES3F1

Module Title:

High Performance Embedded Systems Design

Last Update: February 2025

Lab 4

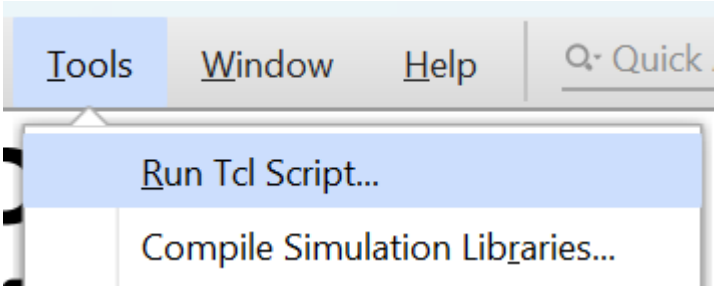
Lab 4 forms the basis for your coursework. We will be using a camera module made by Digilent called the PCam 5C [1]. In this lab we will setup the demo project for this module [2][3]. To simplify the process this Vivado project has been scripted. Feel free to read through the script to help learn how it works. This demo was created using Vitis Classic which is an older version of Vitis that is deprecated but a port to Vitis Unified IDE has not yet been completed.

This project uses the camera connected to the FPGA. Open the case and move the camera module so it is mounted on the top of the case. Please be careful with the flat flex cable. Ensure you remove the plastic lens cap and keep it somewhere safe. If you get distorted video there may be an issue with the cable, please talk to your lab tutor about this.

You need to have at lease 500MB free on your H: drive to do this lab. Please check it before you start doing this lab.

Vivado Setup

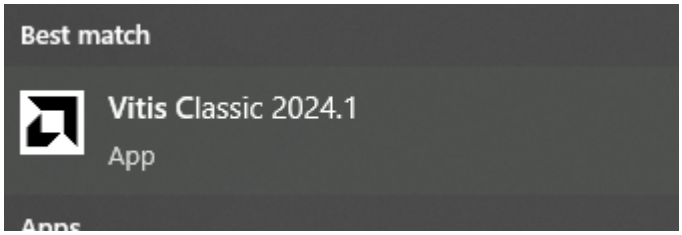
- Open Vivado. In the top menu bar select “Tools->Run TCL Script”.
- In the lab folder select "create_project.tcl” and click “OK”.
  Ensure that the lab folder structure is unchanged and there is no spaces in the file path. This script references the other scripts and files as relative paths from the script location.



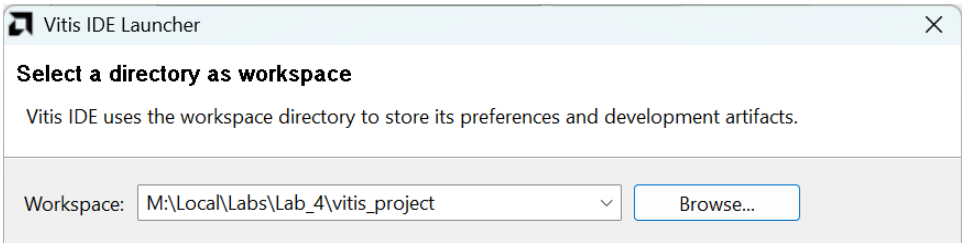
- Wait for the script to finish.
- Generate the bitstream and export hardware including the bitstream.
  You will get five critical errors that can be safely ignored.

Vitis Classic Setup

- Open the Vitis Classic application. Start button -> Vitis Classic



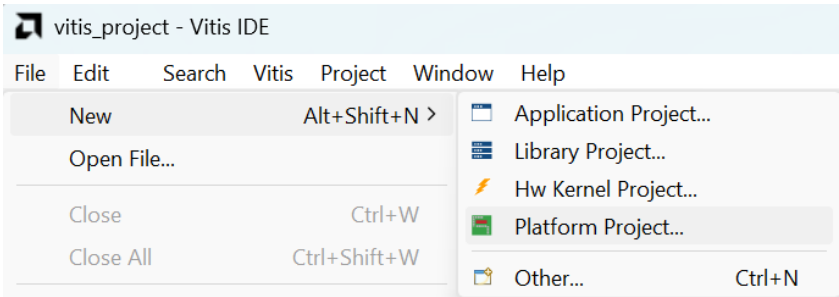
- Create a new folder to use as the workspace and click "launch"



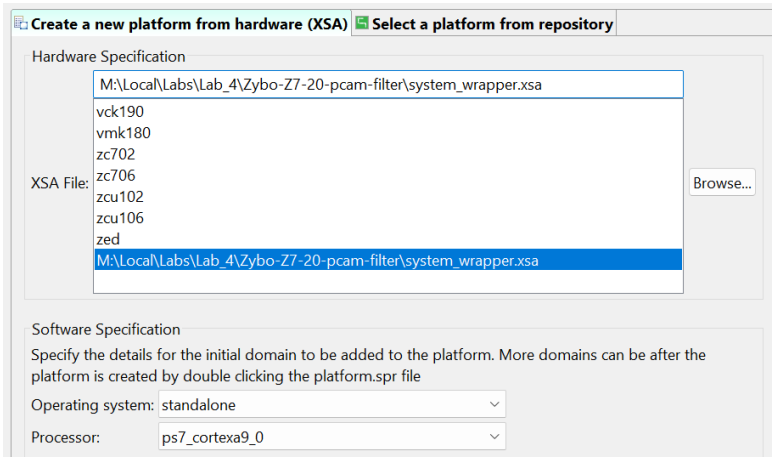
- Vitis classic has been deprecated but the PCam drivers have not been updated so click continue with "classic Vitis"

Platform Project

- Click "File->New->Platform Project"



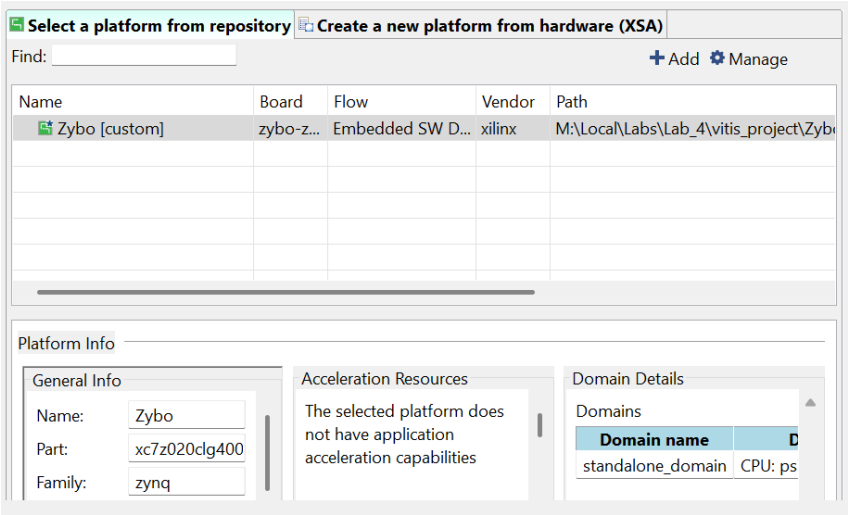
- Set a name
- Click "Browse" next to XSA file and select the exported hardware from the Vivado project.



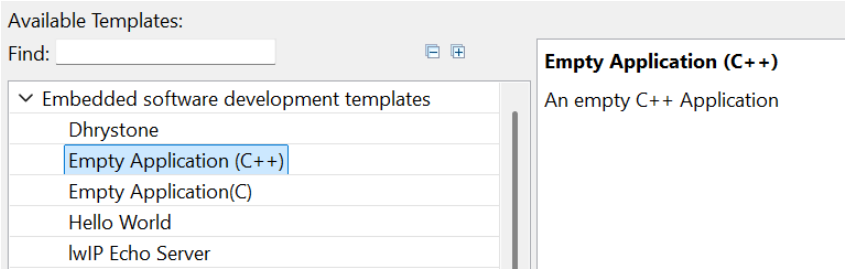
- Click "Finish"

Application Project

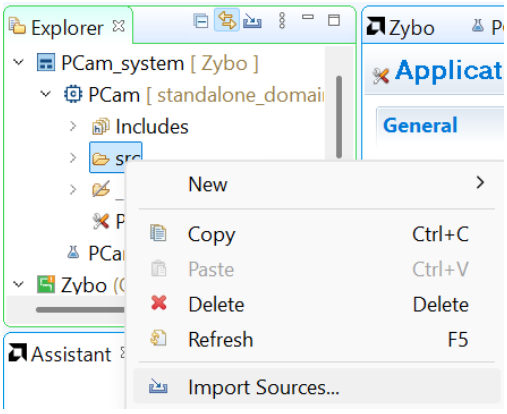
- Click “File->New->Application Project”
- Select the platform you created



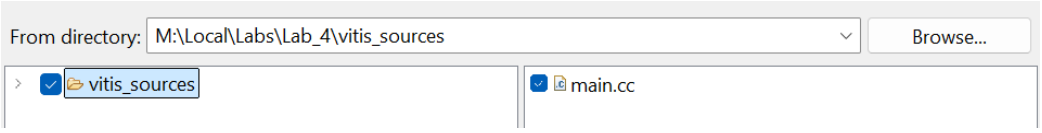
- Set an application name
- Click “Next” until template selection
- Select “Empty Application (C++)” and “Finish”



- In the Explorer window on the left find the application and right click the “src” folder inside it. Then select “Import Sources...”

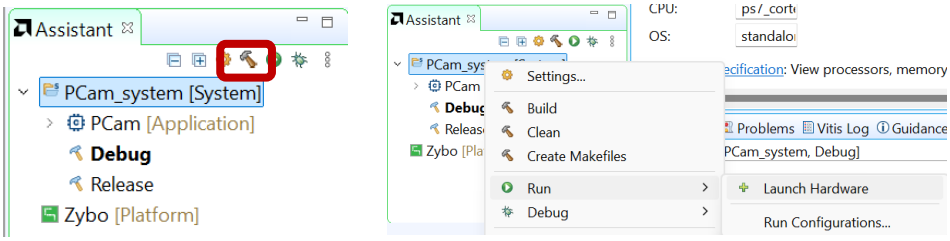


- Browse to the “vitis_sources” directory
- The folder should appear on the left. Ensure that the check box is checked then click “Finish”

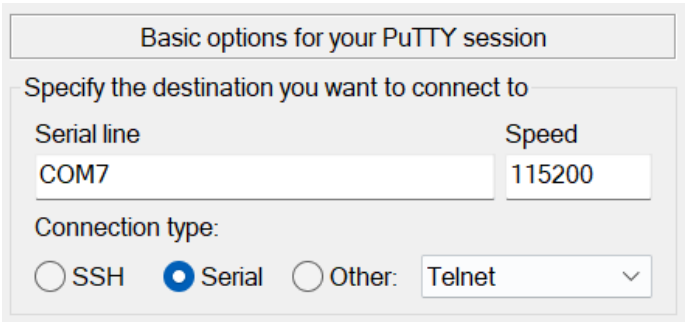


Build and Run the Project

- In the Assistant window on the left, select the "[System]" entry and click the hammer icon in the assistant window header to build the project



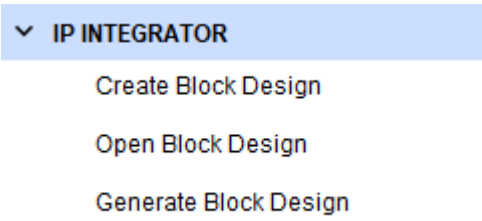
- Open a PUTTY terminal connection to the FPGA serial port as done in previous labs.



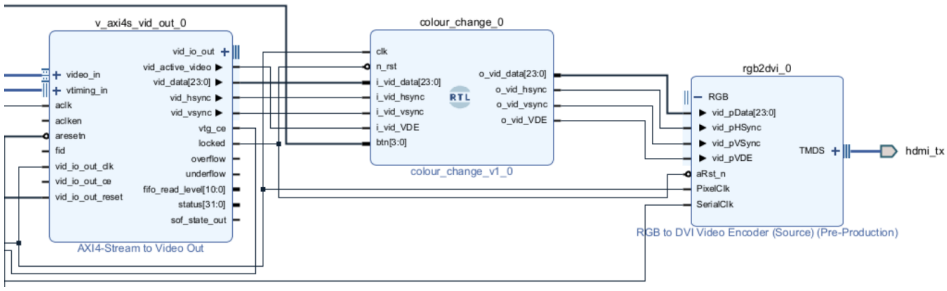
- Right click the "[System]" entry and select "Run->Launch Hardware". This will upload the bitstream and run the application. You should see a video output on the HDMI.

Tasks

1. Have a play with the serial connection to change camera settings. If you look up documentation for the camera sensor or read through its driver code, you may find useful register settings.
2. Add "colour_change.v" as a design source in the Vivado project as you did in last year's module. This module takes in an RGB signal and swaps the red and blue values before outputting the modified signal. This RGB signal holds the data for a single pixel and iterates through the whole image. Have a read through the code to understand how it works.
3. Now open the block design. Under "IP Integrator" select "Open Block Design"

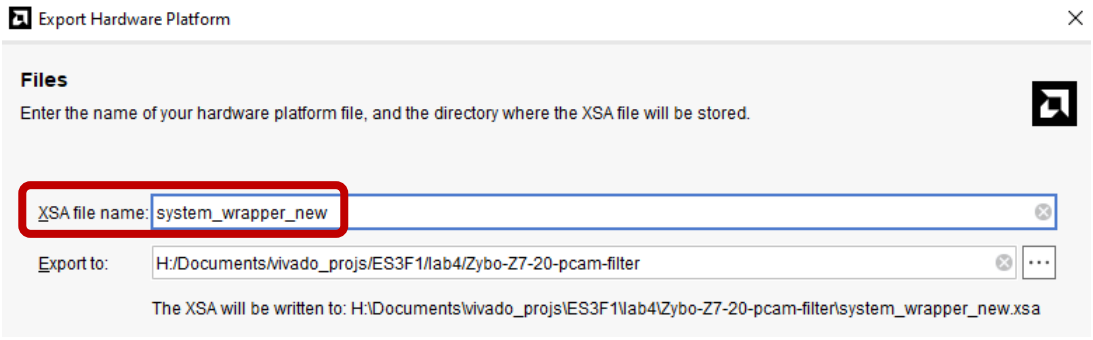


4. In the block diagram, right click and select “Add Module”. You can then add “colour_change”. You can also drag the module into the diagram from the sources list.
5. Connect this module inline between the “AXI4-Stream to Video Out” and the “RGB to DVI Video Encoder” modules by expanding the “vid_io_out” and “RGB” interfaces to access the data, hysnc, vsync and VDE (vid_active_video) signals. Connect “clk” to “PixelClk” and “n_rst” to “aRst_n”.



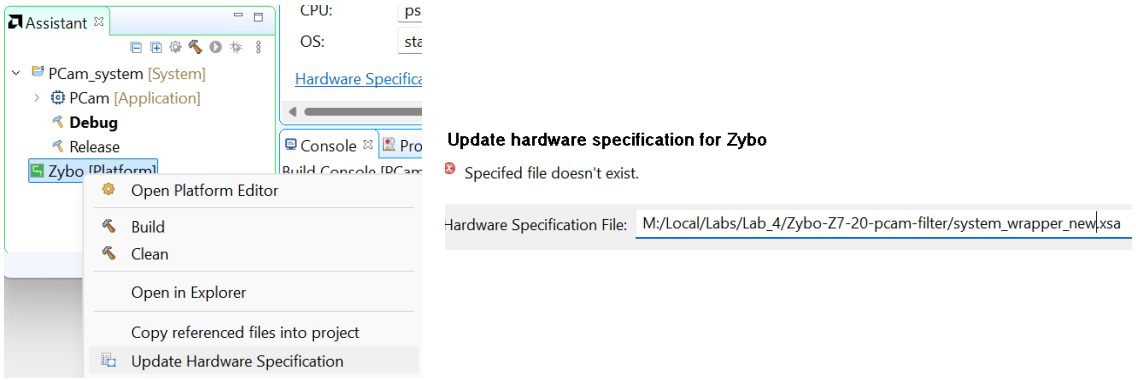
6. Regenerate the bitstream and then export the hardware.

Rename the XSA file to something different to what was used last for the platform. If you update the hardware specification for the project with a file with the same name as used previously it breaks the whole project.

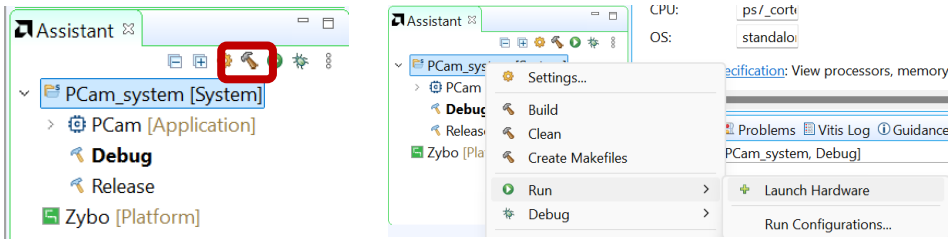


Now go back to **Vitis Classic**. To use the new hardware specification right click on the “[Platform]” entry in the assistant window of **Vitis Classic** and select “Update Hardware Specification”.

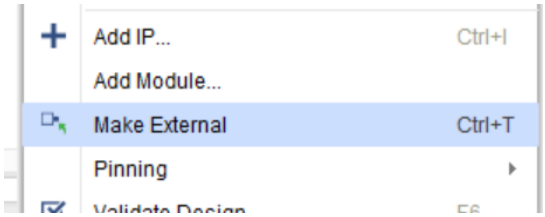
When running always use “Launch Hardware” as if you reuse the older ones it will attempt to upload a bitstream with the old XSA name and fail.



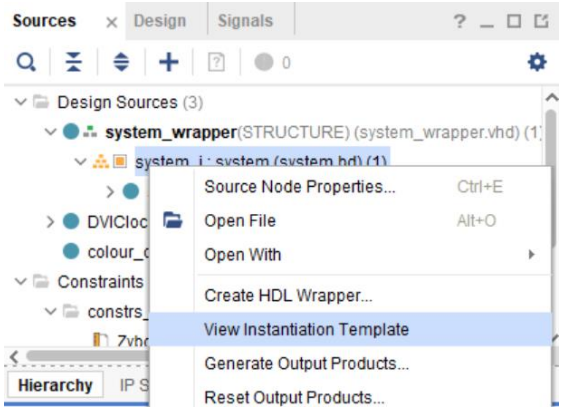
7. You should then rebuild and run the project. You should see the red and blue colours have been swapped.



8. Replace “colour_change.v” with the provided “colour_change_btn.v”. This module has an additional input that is used to enable the colour change. Connect the block as before. Right click the “btn[3:0]” input and select “Make External”.



This should add an external port to the block diagram. Rename the port from “btn_0” to “btn” and in the “ZyboZ7_A” constraints file uncomment the buttons. Save the block diagram and close it to return to the project manager. As we have changed the block diagrams ports the system wrapper needs to be updated. Expand the system wrapper file to show the block diagram file. Right click it and select “View Instantiation Template”.



A new file will open in the code editor. Select everything in this file and copy it. Now open up the current "system_warpper" file and select all the content then past you copied code overwriting everything in the file. This should have added the port. Generate, update and run the project as before. You should now be able to engage the colour swap filter by pressing button 0.

Extension

This lab forms a basis for your coursework. Read through the C++ code used in Vitis Classic to control the camera and try to understand what it is doing. Think about how you could control hardware on the FPGA using this C++ code potentially using AXI Lite registers or DMA. Think about what sort of filters you could apply to the RGB signal from the camera. How could you apply multi-pixel filters like an edge detector which needs multiple pixels as input. If you are interested in using HLS to create filter these resources may be useful:

- https://digilent.s3-us-west-2.amazonaws.com/resources/whitepapers/EmbeddedVisionDemo.pdf
- https://s3-us-west-2.amazonaws.com/digilent-file-share-public/zyboz7_workshop.zip

Resources

[1] - https://digilent.com/reference/add-ons/pcam-5c/reference-manual

[2] - https://digilent.com/reference/programmable-logic/zybo-z7/demos/pcam-5c

[3] - https://digilent.com/reference/learn/programmable-logic/tutorials/zybo-z7-pcam-5c-demo/start